VOICEOPS 2026

LLM and Generative AI Capstone Report

Project	Agentic AI-Based Voice Customer Support System
Organization	ZentroMart (fictional company; synthetic dataset)
Repository	https://github.com/saibalaji2004/voiceops-2026-capstone
Report date	24 October 2026

Executive summary

VoiceOps develops a customer-support assistant for a synthetic Indian e-commerce business. The implementation progresses from a prompt-only baseline to retrieval-augmented generation (RAG), a stateful agent with guarded order and ticket tools, and a voice interface using Gnani speech services. The application combines evidence retrieval, concise policy answers, conversation context, caller verification, tool-backed actions, and generated speech.

The strongest observed result is improved retrieval coverage with top_k=8. The voice pipeline has also completed end-to-end tests on an order-status clarification and a policy question. The evaluation evidence has limitations: saved Step 1/Step 2 answer metrics were generated before the latest retrieval tuning, and the full 23-conversation practice suite has not yet been confirmed as completed.

Goals and scope

- Answer policy questions from the supplied Markdown knowledge base and product catalogue, citing real section IDs where available.
- Keep the prompt-only baseline separate from the RAG chatbot so the two approaches can be compared.
- Support multi-turn follow-ups and require caller verification before revealing protected order details.
- Use mock lookup_order and create_ticket tools and avoid claiming actions unless tool results confirm them.
- Use Gnani for both speech-to-text (STT) and text-to-speech (TTS), as required by the brief.

System flow

Audio file or typed turn	Transcript	Memory, verification, decisions	Knowledge-base sources / order tools	Text, WAV and evidence JSON

Implementation

The capstone brief describes four stages. The project uses existing modules for each stage rather than creating a separate implementation for the voice path.

1. Prompt-only baseline	app/chatbot.py; app/evaluate_step1.py	Calls an LLM with a ZentroMart support prompt, without the main RAG evidence pipeline.	Saved baseline answers and metrics are available in outputs/.
2. RAG	app/rag.py; app/rag_chatbot.py; app/evaluate_step2.py	Indexes 21 Markdown documents and the 70-product catalogue; retrieves top-k evidence and instructs the LLM to answer only from relevant evidence.	Retrieval sweep and saved Step 1 vs Step 2 comparison artifacts are available.
3. Stateful agent	app/step3_agent.py; dataset tools/	Maintains turn context; asks for order IDs, applies verification before disclosure, supports mock order lookup and ticket creation, and routes policy questions to RAG.	Manual order verification and successful ticket creation were exercised. Full practice-conversation evaluation remains to be confirmed.
4. Voice	app/voice.py; run.py	Uses Gnani STT to transcribe input, passes the transcript to the same Step 3 agent and uses Gnani TTS (timbre-v2.5, Kaveri) to save a WAV reply.	Single-file and one-file batch tests succeeded, including a RAG policy question.

Batch runner and evidence outputs

The root-level run.py accepts an input directory, an output directory and a language option. Each successful audio input produces a transcript/reply text file, a spoken WAV response and a JSON evidence record; batch_manifest.json records per-file success/failure metadata. A test using customer_audio/voice_test.wav completed with 1 success and 0 failures. A second test using policy_audio/policy_question.wav.m4a completed with 1 success and 0 failures.

The policy test produced a RAG answer and saved eight sources. After updating collect_sources(), the evidence JSON includes actual retrieved text with section IDs and source filenames, including RET-2 and RET-3. The runner preserves empty source lists for clarification or tool-only turns rather than inventing citations.

Data and safety controls

- Uses the supplied synthetic dataset; no real customer data or third-party call recordings are used.
- Requires phone suffix or delivery pincode verification before order details are disclosed.
- A ticket is treated as created only when the mock tool reports success; ticket creation is not described as proof a human has joined.
- Credentials are configured through environment variables. .env is ignored by Git and .env.example uses placeholders.

Main modules: app/main.py, app/llm_router.py, app/rag.py, app/rag_chatbot.py, app/step3_agent.py, app/voice.py and run.py.

Evaluation results

The following numbers are from saved evaluation artifacts already generated during development. They should be interpreted with their stated denominators and timing, not as a guarantee of production accuracy.

Required phrase coverage	8 / 58 (13.79%)	19 / 58 (32.76%)	Improved by 18.97 percentage points in this saved evaluation.
Checklist pass rate	6 / 50 (12%)	13 / 50 (26%)	More saved answers met the checklist, but overall pass rates remain low.
Forbidden phrase hits	0	0	No hits detected by this specific checker.
Mean gold-evidence recall	Not reported here	47.87%	Measured on saved Step 2 outputs created with earlier retrieval settings.

Retrieval sweep (latest local retriever configuration)

Section recall	76 / 77 = 98.70%	Nearly all expected evidence sections were retrieved across the evaluated questions.
Section precision	76 / 362 = 20.99%	Many returned sections were not among the expected sections, so relevance can still be improved.
Questions with all expected sections	46 / 47 = 97.87%	One evaluated question did not have complete expected-section coverage.

The top_k sweep indicated that top_k=8 is a useful coverage/precision trade-off for this dataset: increasing beyond 8 did not improve the count of questions with all expected sections. The lower precision shows that retrieval ranking remains a tuning target.

Offline audit of saved responses

Unsupported/fabricated citation IDs	0
Real sections cited but not retrieved	1
Answers without bracketed section citations	14
Potentially truncated/empty answer	1
Missing expected sections	32
Product contradictions	0
Decision-handling flags	3

Important evaluation limitation

The answer-comparison and offline-audit results refer to saved outputs generated before the latest retriever tuning. They were not regenerated end-to-end after the most recent changes. The latest retrieval sweep is separate and reflects the updated retriever. The full 23-conversation/96-turn practice suite is not confirmed as fully evaluated; manual tests have covered order ID clarification, caller verification, ticket creation, and an audio policy question.

What went wrong and what to improve

The development process identified several limitations worth addressing before production use or a broader evaluation.

Early RAG answers had incomplete evidence coverage and some unsupported general advice.	Improved retrieval ranking and answer instructions; saved audit identified missing expected sections and citation gaps.	Run a targeted regression set for weak questions and tune ranking without blindly rerunning all 52 LLM cases.
The evidence JSON originally stored metadata but omitted chunk text.	collect_sources() was updated to include chunk/source identifiers and actual text. The policy batch output confirms this content is saved.	Keep evidence attached to every generated RAG answer and add a test that asserts non-empty text for each returned chunk.
Some retrieved chunks are only loosely relevant.	The top_k=8 sweep prioritizes recall but precision remains 20.99%.	Improve reranking and filter loosely related evidence; keep source-precedence and superseded-policy rules intact.
Only a small number of end-to-end voice cases have been executed.	Order-status clarification and policy RAG audio tests succeeded using Gnani STT/TTS.	Run multiple files through the batch command, including unclear speech and a follow-up; inspect text, audio and evidence output for each file.
The full supplied conversation suite has not been confirmed complete.	Manual interaction tests covered core order and ticket flows.	Evaluate all 23 conversation IDs in turn order and record expected decisions, tool calls, and gold evidence coverage.

Final validation status at report preparation

Python syntax compilation	Passed	compileall and py_compile completed without errors.
Dataset validation	Passed	All 9 CSVs, 21 KB documents, keys, order math and practice evidence validated.
Step 3 order verification / ticket action	Manual tests passed	Verified order status after valid pincode; mock ticket created with success confirmation.
Single-file Gnani voice flow	Passed	STT transcript received and TTS WAV saved.
One-file batch runner	Passed	Two sample batch runs recorded 1 success and 0 failures each.
Full practice conversations	Not confirmed	Full 23-conversation / 96-turn evaluation remains a finalization task.

Submission repository

https://github.com/saibalaji2004/voiceops-2026-capstone

Submission package should include the root README.md, .env.example with placeholders, requirements.txt, run.py, source code, report.pdf, demo.mp4 and sample_outputs/. Do not include .env, real API keys, a virtual environment, or a copy of the supplied dataset in the submission ZIP. The README must explain how to place/use the organizer-provided dataset and give the exact batch command.

Conclusion

The project demonstrates the required four-stage architecture and has working local retrieval, guarded tool use and Gnani voice integration. Current metrics show improved saved-answer checklist and phrase coverage from Step 1 to Step 2, and high retrieval recall at top_k=8, with remaining precision and full-conversation validation work clearly identified. The next priority is reproducible batch evaluation and submission packaging.